

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа 1

Выполнил:

Филатов Арсений Сергеевич

Группа К3339

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

1. Цель работы

Освоить практический цикл создания backend-приложения с реляционной базой данных: описание предметной области в виде схемы данных, развёртывание СУБД в контейнере, применение миграций, наполнение тестовыми данными и реализация REST API с документацией и проверкой запросов.

2. Задачи

Спроектировать модель данных предметной области «платформа поиска работы» (пользователи, роли, компании, вакансии, резюме, отклики, справочники).

Развернуть PostgreSQL, связать приложение с БД через переменные окружения.

Описать схему в Prisma, выполнить миграции и согласовать типы/status-поля (в т.ч. перечисления).

Реализовать REST API на стеке Bun + Elysia с группировкой маршрутов под префиксом /api/v1.

Обеспечить регистрацию/вход, выдачу access/refresh токенов, защищённые операции для соискателя и работодателя.

Настроить автоматическую OpenAPI-документацию.

Подготовить сценарий проверки API (вручную через OpenAPI UI, Postman / Collection Runner, при необходимости).

Реализован сервис похожий на классическую job-платформу:

Пользователь принадлежит роли (соискатель, работодатель, администратор в справочнике ролей).

Соискатель имеет профиль, один или несколько резюме с блоками образования, опыта и связью M:N с навыками.

Работодатель связан с компанией через членство (membership); от имени компании создаются вакансии.

Вакансия относится к отрасли и уровню опыта, может содержать набор требуемых навыков, имеет статус жизненного цикла (черновик / опубликована / закрыта) и параметры занятости (полная/неполная ставка, контракт).

Отклик связывает соискателя, вакансию и конкретное резюме; статус отклика поддерживает этапы рассмотрения (новый, просмотрен, отклонён, приглашение).

Ограничение целостности: один отклик соискателя на одну вакансию (@@unique([vacancyId, userId])).

5. Развёртывание базы данных

5.1. Контейнеризация. В `docker-compose.yml` описан сервис `postgres` на образе `postgres:16`, порт `5432`, БД `lab1`, пользователь и пароль заданы через переменные окружения контейнера; данные сохраняются в именованном томе `postgres_data`.

5.2. Подключение приложения. Строка подключения задаётся в `.env` как `DATABASE_URL` (см. `.env.example`).

5.3. Миграции. Схема описана в `prisma/schema.prisma`; применение изменений — команда `bun run db:migrate` (обёртка над `prisma migrate dev`). В ходе работы выполнена начальная миграция и последующая правка перечислений/статусов (отдельная миграция `enums_statuses`).

5.4. Начальные данные. Скрипт `prisma/seed.ts` создаёт роли, тестовых пользователей (соискатель и работодатель), справочники (отрасль IT, уровни `junior/middle`, навыки `TypeScript/PostgreSQL`), компанию с членством работодателя, резюме соискателя, опубликованную вакансию и пример отклика. Запуск: `bun run db:seed`.

6. Реализация REST API

6.1. Точка входа. Приложение собирается в `src/index.ts`: подключается плагин `OpenAPI`, маршруты монтируются в группу `/api/v1`.

6.2. Структура маршрутов (логические модули).

`auth` — `POST /auth/register`, `POST /auth/login`, `POST /auth/refresh`; пароли хэшируются (`bcrypt`), `refresh`-токен сохраняется в таблице `RefreshToken`.

`users` — `GET/PATCH /me` (текущий пользователь, частичное обновление полей).

`catalog` — `GET /industries`, `GET /experience-levels`, `GET /skills` (пагинация и поиск по навыкам).

`job-seeker` — профиль соискателя, `CRUD` резюме, блоки образования и опыта, замена набора навыков резюме.

`companies` — создание компании (для не-кандидатов), карточка компании, обновление при наличии `membership`, список членств текущего пользователя.

`vacancies` — публичный поиск с фильтрами (`industryId`, `experienceLevelId`, зарплатный коридор, строка `q`), детальная карточка; для работодателя — создание и изменение вакансий от имени компании, управление навыками вакансии, удаление.

`applications` — создание отклика кандидатом с проверкой владения резюме; список «мои отклики»; для работодателя — список откликов по вакансии и смена статуса отклика.

6.3. Авторизация в коде. Функция `getAuthUser` из заголовка `Authorization: Bearer ...` извлекает идентификатор пользователя и загружает запись из БД (в учебной версии допускается упрощённый формат токена).

6.4. Контракты и ошибки. Ответы описаны через схемы `Elysia/TypeBox` в `src/schemas/index.ts`; ошибки унифицированы (`code`, `message`, опционально `details`).

6.5. Документация. После запуска `bun run dev` доступны интерактивная страница `/openapi` и машиночитаемый JSON `/openapi/json`. В спецификации указаны теги по разделам API и схема `bearerAuth`.

7. Порядок запуска проекта (локально)

```
bun install
```

```
bun run db:up
```

```
bun run db:migrate --name init # при первом развёртывании; далее по необходимости
```

```
bun run db:seed
```

```
bun run dev
```

Проверка: открыть `http://localhost:3000/openapi`, выполнить типовые запросы или прогнать коллекцию Postman (сценарий «регистрация → вход → каталог → поиск вакансий → карточка → профиль и резюме → отклик → список откликов»).

8. Результаты

Развёрнута реальная СУБД PostgreSQL в Docker, данные сохраняются между перезапусками (том).

Схема БД отражает связи сущностей job-платформы, включая M:N для навыков и уникальность отклика на вакансию.

Реализован REST API с версионированием через префикс `/api/v1`, разграничением операций кандидата и работодателя.

Настроена автогенерация OpenAPI из кода маршрутов.

Подготовлены тесты в Postman для сквозного сценария основного пользовательского процесса соискателя.

9. Выводы

В ходе лабораторной работы отработан полный цикл: от моделирования данных и миграций до работающего HTTP API поверх PostgreSQL. Использование Prisma ускорило согласование схемы БД с TypeScript-кодом, а Elysia с плагином OpenAPI снизило стоимость сопровождения контракта API. Дальнейшее развитие: усиление безопасности (полноценные JWT/подпись, отзыв сессий), пагинация и индексы под нагрузочные запросы, интеграционные тесты помимо ручных/Postman.